Acuity Analytics — Cloud Engineer L2 Interview Preparation

AWS + Azure | Terraform | CI/CD | ArgoCD | Helm | Kubernetes | Security | Networking | SRE | DR | Databricks | Kafka | Capital Markets

Role basis: Cloud Engineer, 8+ years, AI/Data & Digital, Chennai. Prepared from the supplied Acuity JD and the L2 preparation discussion.

> **Interview approach:** Speak naturally. For production incidents use: Impact → Triage → Metrics/Logs → Isolate → Mitigate → Recover → RCA → Prevent.

L2 Priority Areas

#	Area
1	Terraform production architecture/state
2	AWS + Azure networking/private connectivity
3	Kubernetes troubleshooting/security
4	CI/CD, Helm and ArgoCD/GitOps
5	IAM, secrets and encryption
6	Observability, SLOs and incident management
7	DR, RTO/RPO and resilience
8	Databricks + Kafka/event-driven fundamentals
9	Cost optimization/capacity
10	Capital-markets production/change control

1. Introduction & Project

1. Tell me about yourself.

I have 11+ years of experience in SRE, DevOps and cloud engineering, primarily across Azure and AWS. In my recent Microsoft Azure Synapse Analytics project, I worked on production reliability, P1/P2 incidents, monitoring, SLI/SLOs, observability, alert optimisation, CI/CD and capacity planning. I have hands-on experience with Kubernetes, Docker, Terraform, Jenkins, GitHub Actions, Azure DevOps, ArgoCD, Helm, Prometheus and Grafana. My strength is combining infrastructure automation with production operations, including security, reliability and operational readiness.

2. What was your responsibility in the Synapse project?

My primary responsibility was production reliability. I handled P1/P2 incidents, monitored platform health, worked with SLI/SLOs and observability, optimized alerts and supported capacity and reliability improvements. I also collaborated with engineering teams on CI/CD and recurring production issues.

2. AWS + Azure Cloud Architecture

3. How do you design a platform consistently across AWS and Azure?

I define common platform standards for networking, IAM, logging, monitoring, encryption, tagging and deployment. Then I map those standards to native services in each cloud. Terraform can provide a common IaC layer while cloud-native services are used where they add value. I keep reusable modules separate from environment-specific configuration.

4. Explain AWS VPC architecture.

I start with a VPC spanning multiple availability zones. Public subnets are used only where required, while application and database workloads remain private. Private workloads use controlled outbound connectivity, and route tables, security groups and NACLs enforce traffic controls. For sensitive services I prefer private endpoints/VPC endpoints.

5. Azure VNet vs AWS VPC?

They provide similar network isolation but use different implementations. AWS uses VPCs, subnets, route tables, security groups and NACLs; Azure uses VNets, subnets, route tables, NSGs and private endpoints. The common principles are segmentation, controlled routing, least privilege and private connectivity.

6. How would you securely connect AWS and Azure?

I first determine whether application-level or network-level connectivity is required. For private connectivity, options include site-to-site VPN or dedicated connectivity such as Direct Connect and ExpressRoute with an appropriate interconnection design. I ensure non-overlapping CIDRs, controlled routing, firewall rules, DNS resolution and redundancy.

3. Terraform / IaC

7. Design a production Terraform repository.

I separate reusable modules from environment-specific configuration. Modules can contain networking, Kubernetes, IAM and monitoring patterns, while dev, test and production directories provide their configurations. State is remote, encrypted and locked. CI performs format, validation, scanning and plan; production apply requires controlled approval.

8. Terraform plan wants to destroy 50 production resources. What do you do?

I stop the apply immediately. I inspect the plan and determine why resources are being replaced or deleted. I compare configuration, state and actual resources and check provider/module changes. If it is state drift or resource addressing, I correct it safely using state operations or import rather than destroying production.

9. What if Terraform state is corrupted?

I stop further Terraform operations, inspect the remote backend and identify the last known-good state. If backend versioning is available, I recover the appropriate version and run a careful plan against actual infrastructure. I would never recreate production infrastructure simply because state is damaged.

10. Module vs resource?

A resource represents an individual infrastructure object, while a module is a reusable collection of resources implementing a higher-level pattern. Enterprise platforms can use modules for networking, Kubernetes or monitoring, with environment-specific values outside the modules.

11. How do you protect secrets in Terraform?

I avoid hardcoded secrets and prefer managed identities, IAM roles, Key Vault or Secrets Manager. A sensitive Terraform variable can still exist in state, so the remote state backend also needs encryption, access control, auditing and restricted permissions.

4. CI/CD, Helm & ArgoCD

12. Explain your ideal CI/CD architecture.

CI starts with Git checkout, validation, tests, security scanning and artifact/container creation. The immutable artifact is stored in an artifact repository. CD promotes that artifact through environments using controlled deployment. For Kubernetes, ArgoCD reconciles desired state from Git with the cluster. Production releases include health validation and rollback.

13. Why use Jenkins/GitHub Actions/Azure DevOps with ArgoCD?

They solve different parts of delivery. Jenkins, GitHub Actions or Azure DevOps can perform CI such as build, test, scanning and publishing. ArgoCD focuses on GitOps-based continuous delivery into Kubernetes. CI can build an immutable image and update deployment configuration in Git, while ArgoCD reconciles it into the cluster.

14. What if someone manually changes an ArgoCD-managed deployment?

The cluster has drifted from the Git desired state. ArgoCD detects the difference and, depending on sync policy, reports or reconciles it. In production I prefer Git to remain the source of truth so manual changes do not become undocumented configuration.

15. Helm vs raw Kubernetes YAML?

Raw YAML is simple but becomes repetitive for environment-specific configuration. Helm provides templating and reusable charts with values. I keep chart structure reusable and put environment-specific configuration in values rather than duplicating manifests.

16. How do you rollback an ArgoCD deployment?

I confirm the deployment version and production impact, then revert the Git change or restore the previous known-good configuration. ArgoCD reconciles the cluster back to the desired version. I verify pod health, error rate and latency before declaring recovery.

5. Kubernetes L2

17. Pod is Running but application is unavailable. What do you check?

I don't assume Running means healthy. I check readiness, Service and endpoints, then ingress/load-balancer configuration and DNS. I verify Service selectors match pod labels and inspect network policies and application logs. I test connectivity from inside the cluster to isolate application, Service, networking or ingress issues.

18. Service has no endpoints. Why?

Typical causes are a selector that does not match pod labels, pods that are not Ready, namespace mismatch, or port/targetPort issues. I check endpoints, compare selectors and labels, inspect readiness and validate Service ports and deployment status.

19. Pod is OOMKilled.

I check memory usage, requests/limits and application logs. I determine whether the workload legitimately needs more memory or has a leak/abnormal behaviour. Increasing the limit can be a short-term mitigation if capacity allows, but I then investigate the underlying cause and tune the application or resources.

20. Kubernetes node CPU is 95%.

I identify whether usage is normal growth or an abnormal workload. I inspect node and pod consumption, requests/limits, HPA behaviour and cluster capacity. If demand is legitimate I scale workloads/nodes; if abnormal I isolate the workload and mitigate it.

21. How would you secure AKS/EKS?

I use RBAC and least privilege, non-root containers, restricted security contexts and network policies. I use private API access where required, integrate secrets with Key Vault or Secrets Manager, scan images and enforce admission policies. I

also enable audit logging, monitoring and regular patching.

6. Cloud Security

22. IAM vs RBAC?

IAM is the broader identity and access-control model; RBAC is role-based permission assignment. Azure RBAC controls resource access, while AWS IAM policies and roles define permissions. I apply least privilege, separation of duties and avoid long-lived credentials.

23. How do you implement secrets management?

I don't store production secrets in Git or container images. I use Azure Key Vault or AWS Secrets Manager, with managed identities or IAM roles where possible. Access is least-privilege and audited, with rotation according to security requirements.

24. Encryption at rest vs in transit?

Encryption at rest protects stored data such as databases, storage and disks. Encryption in transit protects communications, typically using TLS. For sensitive financial workloads I ensure both, with controlled key management, rotation and access.

7. Networking

25. Private Endpoint isn't working. How do you troubleshoot?

I start with DNS because private endpoint issues frequently involve name resolution. I verify the hostname resolves to the private IP. Then I check routing, NSGs/security groups, firewall rules and endpoint state. I test TCP connectivity and inspect application logs to isolate DNS, routing, security or authentication.

26. What is DNS split-horizon?

It means the same hostname can resolve differently depending on where the request originates. Internal clients can resolve a service to a private IP while external clients resolve it differently. This is important for private endpoints and private DNS zones.

8. Observability & SRE

27. What would you monitor for a trading platform?

At infrastructure level I monitor CPU, memory, network, storage and cluster health. At application level I monitor request rate, error rate, latency and dependencies. For event-driven systems I additionally monitor message throughput, queue depth, consumer lag, processing latency and failed messages. I map signals to SLOs and business impact.

28. How do you reduce alert fatigue?

I classify alerts by severity and customer impact, remove alerts that do not require action, deduplicate related alerts and use sustained thresholds. I prefer SLO-based alerts for critical services and review false positives and incident history.

29. Explain SLI, SLO and SLA.

An SLI is a measured reliability indicator such as availability or latency. An SLO is the target for that indicator. An SLA is an external contractual commitment, often with consequences if missed. I use SLOs and error budgets to balance reliability with delivery.

30. What is an error budget?

It represents the amount of unreliability allowed by an SLO. For a 99.9% availability target, the remaining 0.1% is effectively the error budget. If it is consumed too quickly, reliability work should take priority and releases may need additional controls.

9. Databricks

31. What is Databricks?

Databricks is a cloud data and analytics platform built around Spark and used for data engineering, analytics and machine learning. From a cloud-engineering perspective I would focus on workspace networking, identity, storage access, compute, secrets, monitoring and deployment. I would not claim deep application-development expertise where I don't have it.

32. How would you secure Databricks?

I start with identity and network isolation, using private connectivity where required. Cloud storage access should use managed identities or controlled service principals. I apply least privilege to workspace, data and compute access, encrypt data, manage secrets centrally and monitor access and administrative activity.

10. Kafka / Event Driven

33. What is Kafka?

Kafka is a distributed event-streaming platform where producers publish messages to topics and consumers read them. Topics are partitioned for scalability and consumer groups share processing. From an SRE perspective I monitor broker health, partition distribution, throughput, consumer lag, replication and failures.

34. What is consumer lag?

Consumer lag indicates how far a consumer group is behind the latest messages. Increasing lag can mean consumers cannot keep up. I correlate lag with consumer CPU, processing latency, partition distribution and incoming throughput before scaling.

35. Kafka vs MQ?

Kafka is optimized for distributed event streaming, high throughput and retaining events for consumers. Enterprise MQ systems are often used for reliable point-to-point or messaging patterns. The choice depends on delivery semantics, throughput, replay, ordering and integration requirements.

11. DR & Resilience

36. Explain RTO and RPO.

RTO is the maximum acceptable time to restore service after disruption. RPO is the maximum acceptable amount of data loss measured in time. For example, a 30-minute RTO means recovery within 30 minutes, while a 5-minute RPO means losing no more than roughly five minutes.

37. How would you design DR for a critical financial application?

I start with business-defined RTO and RPO. Based on those requirements I choose backup recovery, warm standby or active-active. Critical data needs appropriate replication and backup, while infrastructure should be reproducible with Terraform. DNS, networking, IAM and secrets also need recovery coverage. Finally, I regularly test failover.

38. DR exists but has never been tested. What do you do?

I treat it as unvalidated. I define expected RTO/RPO, execute a controlled DR exercise and measure recovery time. The test should validate dependencies, DNS, credentials, data consistency, monitoring and failback. Gaps become remediation actions.

12. Cost & Capacity

39. How do you reduce cloud cost without hurting reliability?

I start with visibility into cost drivers, then identify idle resources, oversized instances, unattached storage, inefficient data transfer and unnecessary non-production runtime. I evaluate committed capacity for predictable workloads. I never reduce resources blindly; I validate utilization against performance and SLOs.

13. Linux & Scripting

40. Server disk is 95% full. What do you do?

I identify the filesystem and top consumers using df, du and find. I check whether growth is from logs, temporary files, application data or deleted files held by processes. I avoid blindly deleting production files and use approved rotation/cleanup. After freeing space I verify health and investigate the growth.

41. How would you automate log cleanup?

I would parameterize the directory and retention period, validate inputs, log actions and run it through cron or an approved scheduler. Where applicable I prefer logrotate rather than custom deletion because it is a standard rotation and retention mechanism.

14. Production Scenarios

42. Production API is down. Walk me through it.

I establish impact and scope — customers, regions and services. Then I check dashboards, alerts, logs and recent changes. I isolate application, infrastructure, network or dependency causes. I prioritize mitigation using rollback, failover or scaling where appropriate. After recovery I perform RCA and preventive actions.

43. CPU, memory and network are normal, but users report slowness.

I don't conclude infrastructure is healthy. I check customer-facing P95/P99 latency, traces and downstream dependency latency. I inspect database queries, connection pools, external APIs, DNS and network latency and correlate with recent changes.

44. Latency increased immediately after deployment.

I correlate timing with the deployment and compare the new version with the previous known-good version. I check errors, latency, resources and dependencies. If the deployment is the likely cause and customers are impacted, I prioritize rollback and investigate after service is stable.

45. Kubernetes cluster is healthy but application is unavailable.

I work from the outside in: DNS, load balancer/ingress, Service, endpoints, pods and application. I check whether traffic reaches ingress, whether the Service has healthy endpoints and whether readiness probes pass.

46. One availability zone fails. What happens?

A highly available design should redistribute traffic to healthy zones and have enough replicas and capacity elsewhere. I verify load-balancer behaviour, Kubernetes scheduling, database replication and remaining-zone capacity.

15. Capital Markets

47. Trading application needs deployment during market hours. What do you consider?

I first understand change policy and service criticality. For a front-office trading system I avoid unnecessary risk during sensitive periods and use approved change windows unless an incident or critical business requirement demands otherwise. If deployment must happen, I use progressive rollout, validation, health checks and tested rollback.

48. Why is security especially important in capital markets?

These environments handle sensitive financial, client and trading information and often operate under strict regulatory and audit requirements. Cloud engineering therefore needs strong identity controls, encryption, network segmentation, secrets management, audit logging, controlled changes and least privilege.

49. Application team requests production admin access. What do you do?

I don't provide broad permanent admin access for convenience. I understand the exact operation and grant minimum required permission. Where possible I use temporary privileged access with approval and auditing. This supports least privilege and separation of duties.

16. Senior Judgment

50. How is SRE different from DevOps?

DevOps is broadly a culture and set of practices connecting development and operations, while SRE applies software-engineering principles to operations and reliability. In practice that means defining SLOs, measuring reliability, automating operations, improving observability and reducing toil while enabling safe delivery.

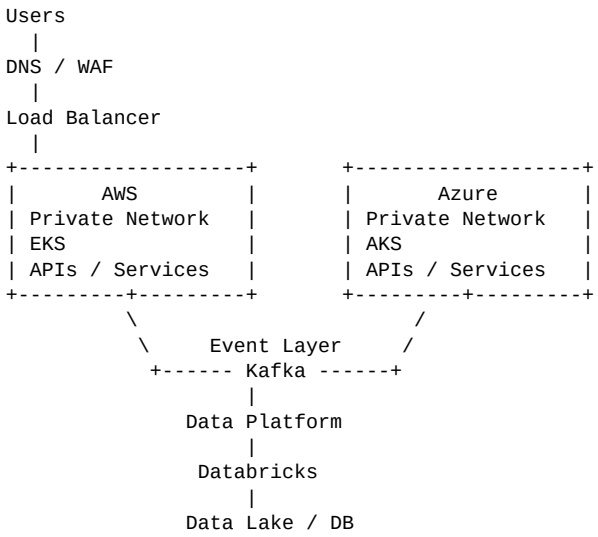
51. What if you haven't worked extensively with Databricks or Kafka?

I would be transparent rather than overstate experience. I can explain the platform concepts and operational aspects I understand, while clearly identifying where hands-on experience is limited. I would then explain how I would approach the architecture using established production engineering principles.

17. L2 Architecture Scenario

Design a secure cloud platform for a global capital-markets application running across AWS and Azure.

Start with requirements, then architecture, security, automation, observability, resilience and cost.



Security: IAM/RBAC, managed identities, secrets management, encryption, private endpoints, network segmentation and audit logging.

IaC: Terraform reusable modules, remote state, CI validation/plan, approvals and controlled production apply.

CI/CD: Git → build/test/scan → immutable artifact → Helm/Git deployment configuration → ArgoCD → Kubernetes → monitoring.

Observability: Metrics, logs, traces, SLOs, actionable alerts and service dashboards.

Resilience: Multi-AZ, regional DR where required, RTO/RPO, backups and regular failover testing.

Operations: Capacity planning, cost optimization, incident management, runbooks and automation.

18. Rapid-Fire Revision

Term	Quick answer
Idempotency	Repeated execution produces the same desired final state.
Horizontal scaling	Add more instances/pods.
Vertical scaling	Increase resources of an existing instance/workload.
Blue-green	Two environments; validate inactive environment, then switch traffic.
Canary	Release to a small percentage first, monitor, then increase.
5xx	Server-side failure; investigate application, dependencies and infrastructure.
4xx	Generally request/client-side issue such as auth, validation or not-found.
Helm	Kubernetes packaging and templating tool.
ArgoCD	GitOps continuous delivery and Kubernetes desired-state reconciliation.
Observability	Understanding system behaviour through metrics, logs and traces.

19. Questions to Ask the Interviewer

• For this capital-markets modernization program, how are responsibilities divided between Cloud Engineering, DevOps, Architecture and application teams?

• Is the target platform primarily multi-cloud by design, or are AWS and Azure being used for different workloads?

• How are you currently implementing GitOps with ArgoCD, and how much of the environment is already managed through Terraform?

• What is the most important reliability or platform challenge for this role during the first six months?

20. Final Answer Strategy

Troubleshooting: Impact → Triage → Metrics/Logs → Isolate → Mitigate → Recover → RCA → Prevent.
Architecture: Requirement → Architecture → Security → Automation → Observability → Failure scenario → Recovery → Cost.

Unfamiliar technology: Be transparent about the boundary of hands-on experience, then explain the relevant production/SRE approach without inventing experience.

Key L2 principle: Demonstrate production judgment, not definitions alone. Tie decisions to reliability, security, operational risk, recoverability and business impact.